



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Selected Topics in Cryptography

Reporte Práctica 05:

“ECDSA with standard elliptic curves”

Alumno:

Sigala Morales Said

2022630413

Grupo: 7CM4

Maestro: Sandra Diaz Santiago

Fecha de realización: 26 de Marzo de 2025

Fecha de entrega: 02 de Abril de 2025



DESCRIPCIÓN GENERAL

En esta práctica se implementaron las funciones necesarias para la firma digital mediante el algoritmo ECDSA, utilizando la librería cryptography de Python. Se buscó que las claves y firmas fueran almacenadas en archivos separados, codificadas en base64, permitiendo también el uso de distintas curvas elípticas del estándar NIST.

GENERACIÓN DE LLAVES

Se creó una función que genera un par de llaves (privada y pública) para ECDSA, usando las curvas P-256, P-384 y P-521. Estas llaves se guardan en archivos .pem separados y sin cifrado, para facilitar la prueba de las siguientes funciones.

```
1 def generate_key_pair(curve_name: str) -> None:
2     private_key_path = os.path.join(OUTPUT_DIR, f"private_key_{curve_name}.pem")
3     public_key_path = os.path.join(OUTPUT_DIR, f"public_key_{curve_name}.pem")
4
5     curve_map = {
6         'P-256': ec.SECP256R1(),
7         'P-384': ec.SECP384R1(),
8         'P-521': ec.SECP521R1(),
9     }
10    if curve_name not in curve_map:
11        raise ValueError(f"Curva {curve_name} no soportada.")
12
13    private_key = ec.generate_private_key(curve_map[curve_name])
14    public_key = private_key.public_key()
15
16    pem_private = private_key.private_bytes(
17        encoding=serialization.Encoding.PEM,
18        format=serialization.PrivateFormat.PKCS8,
19        encryption_algorithm=serialization.NoEncryption()
20    )
21    pem_public = public_key.public_bytes(
22        encoding=serialization.Encoding.PEM,
23        format=serialization.PublicFormat.SubjectPublicKeyInfo
24    )
25
26    with open(private_key_path, "wb") as f:
27        f.write(pem_private)
28    with open(public_key_path, "wb") as f:
29        f.write(pem_public)
30
31    print(f"[OK] Clave privada guardada en {private_key_path}")
32    print(f"[OK] Clave pública guardada en {public_key_path}")
```



- Se usa **ec.generate_private_key** para generar la llave.
- Las claves se serializan usando el formato **PEM**.
- Cada clave se guarda en el directorio practica05.

La función `generate_key_pair` se encarga de generar un par de claves para el algoritmo ECDSA utilizando una curva elíptica especificada por el usuario. Primero, define las rutas donde se guardarán los archivos de la clave privada y pública, dentro de una carpeta llamada `practica05`. Después, establece un diccionario que asocia nombres de curvas estándar (P-256, P-384, P-521) con sus respectivas implementaciones en la librería `cryptography`. Si el nombre de la curva proporcionado no está en ese diccionario, lanza un error informando que la curva no está soportada.

FIRMA DE MENSAJES

Se implementó una función que firma mensajes con la llave privada. El algoritmo de firma usado es ECDSA con SHA-256 como función hash. El (r,s) codificada en base64 se guarda en un archivo `signature.txt`.

```
1 def sign_message(private_key_path: str, mensaje: bytes) -> tuple:
2
3     with open(private_key_path, "rb") as f:
4         private_key = serialization.load_pem_private_key(f.read(), password=None)
5     der_signature = private_key.sign(
6         data=mensaje,
7         signature_algorithm=ec.ECDSA(hashes.SHA256())
8     )
9     r, s = decode_dss_signature(der_signature)
10
11     with open(os.path.join(OUTPUT_DIR, "mensaje.txt"), "wb") as f:
12         f.write(mensaje)
13     return (r, s)
```

La función `sign_message` se encarga de generar una firma digital para un mensaje utilizando ECDSA junto con la función hash SHA-256. Esta función recibe como parámetros la ruta del archivo que contiene la clave privada en formato PEM y el mensaje que se desea firmar, el cual debe estar codificado en bytes.



Primero, se abre el archivo de la clave privada en modo lectura binaria y se carga usando la función `load_pem_private_key` de la biblioteca `cryptography`. Esta función interpreta el contenido del archivo y devuelve un objeto de clave privada listo para ser utilizado.

- Se carga la clave privada desde archivo.
- Se usa **`private_key.sign()`** con **`ECDSA(SHA256())`**.
- La firma se convierte a **`base64`** y se almacena.



VERIFICACIÓN DE FIRMAS

Se implementó una función que verifica una firma ECDSA. Recibe el mensaje original, el archivo con la clave pública y la firma en base64. Devuelve un valor booleano indicando si la firma es válida.

```
1 def manual_verify_signature(public_key_path: str, rs: tuple) -> bool:
2
3     with open(public_key_path, "rb") as f:
4         public_key = serialization.load_pem_public_key(f.read())
5         pub_nums = public_key.public_numbers()
6         Q = (pub_nums.x, pub_nums.y)
7
8     mensaje_path = os.path.join(OUTPUT_DIR, "mensaje.txt")
9     with open(mensaje_path, "rb") as f:
10         m = f.read()
11
12     e = int.from_bytes(hashlib.sha256(m).digest(), byteorder='big') % n
13     (r, s) = rs
14
15     if not (1 <= r < n and 1 <= s < n):
16         return False
17
18     w = mod_inv(s, n)
19     u1 = (e * w) % n
20     u2 = (r * w) % n
21
22     point1 = scalar_mult(u1, G)
23     point2 = scalar_mult(u2, Q)
24     if point1 is None:
25         Xp = point2
26     elif point2 is None:
27         Xp = point1
28     else:
29         Xp = point_add(point1, point2)
30
31     if Xp is None:
32         return False
33
34     x_coord = Xp[0] % n
35     return x_coord == r
```

La función `manual_verify_signature` implementa el proceso de verificación de una firma digital generada con ECDSA, de forma explícita y paso a paso,

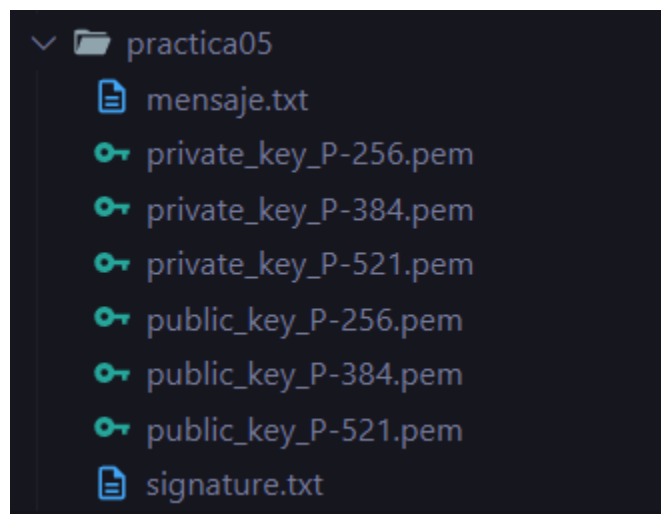


sin depender de funciones automáticas de la biblioteca. Esta función toma como entrada la ruta del archivo que contiene la clave pública y la tupla (r, s) correspondiente a la firma, y devuelve un valor booleano que indica si la firma es válida o no.

EJEMPLOS DE EJECUCIÓN

```
5. Salir
Seleccione una opción: 4
Generando pares de llaves para las curvas: ['P-256', 'P-384', 'P-521']
[OK] Clave privada guardada en practica05\private_key_P-256.pem
[OK] Clave pública guardada en practica05\public_key_P-256.pem
[OK] Clave privada guardada en practica05\private_key_P-384.pem
[OK] Clave pública guardada en practica05\public_key_P-384.pem
[OK] Clave privada guardada en practica05\private_key_P-521.pem
[OK] Clave pública guardada en practica05\public_key_P-521.pem
[OK] Se han generado las llaves.
```

Generación de llaves con distintas curvas para corroborar el buen funcionamiento de la generación de claves.



Salida de las llaves y el archivo de firma generada.